GRAFANA

Visualizando Dados

Instrutor: Lyjor Amorim de Barros

16h • 4 noites • 18:30 às 22:30

Green Tecnologia

Material do aluno — uso pessoal, sem redistribuição

CÓPIA DO ALUNO

Curso

Básico → Intermediário

Quatro noites de imersão, com teoria, demonstração ao vivo e laboratórios guiados. Material de apoio, caderno de exercícios e ambiente Docker pronto para usar.

Conteúdo programático

- Observabilidade e os 3 pilares
- Grafana 11 e navegação na UI
- Prometheus + PromQL
- Visualizações, variáveis e alertas
- Instalação em Ubuntu (prática)
- Provisionamento e multi-stack

Quem está conduzindo o curso

32 anos

LPI

MCT

Power BI

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Para quem é este curso

- Arquitetos de informação e de soluções
- Service Delivery Managers e líderes de operação
- Analistas de dados e observabilidade
- Equipes de operações, SRE, performance e NOC
- Profissionais que precisam apresentar dados a stakeholders técnicos e de negócio

Curso básico-intermediário. Vamos do zero até a configuração de alertas e provisionamento de dashboards em ambiente de produção. Não exige conhecimento prévio de Grafana, mas o aluno precisa estar confortável com terminal Linux e conceitos básicos de redes.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

RESULTADOS DE APRENDIZAGEM

O que você vai dominar

1 **Observabilidade**

Os 3 pilares — métricas, logs e traces — e por que o Grafana é a peça central.

2 **Interface**

Navegar com confiança: menus, dashboards, Explore, Alerting e configurações.

3 **Queries**

Construir consultas em PromQL e em SQL para extrair dados significativos.

4 **Visualizações**

Escolher e customizar o painel certo para cada cenário, do stat ao heatmap.

5 **Alertas**

Configurar alertas Prometheus e Grafana, com canais de notificação.

Em todas as noites, ao menos 50% do tempo é prática hands-on. Você sai do curso com dashboards e alertas funcionando na sua própria máquina.

CÓPIA DO ALUNO

ESTRUTURA

Roteiro das quatro noites

Dia	Tema	Teoria + demonstração (18:30–20:30)	Laboratório (20:30–22:30)
1	Fundamentos e navegação	Observabilidade, Grafana 11, Prometheus, Node Exporter, PromQL básico	Subir Docker, navegar no Grafana, criar primeiro dashboard
2	Visualizações, variáveis, alertas	Tipos de painel, variáveis de template, alertamento Prometheus e Grafana	Dashboard dinâmico com variável, alerta FIRING, painel de containers
3	Instalação no Ubuntu	Repositórios APT, systemd, segurança, arquivos de configuração	Execução guiada dos 6 scripts comentados, do zero ao Grafana rodando
4	Temas, provisionamento, multi-stack	Theming, file-based provisioning, plugins, Loki, Tempo	Provisionar dashboard por arquivo, instalar plugin, adicionar datasource

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

DIA 01

Fundamentos e navegação

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Grafana: Visualizando Dados

O que veremos na primeira noite

1 — Observabilidade

O que é, por que importa, os 3 pilares.

2 — Grafana

História, licenciamento, casos de uso.

3 — Stack local

Grafana + Prometheus + Node Exporter em Docker.

4 — UI

Menus, dashboards, Explore, configurações.

5 — Hands-on

Subir, navegar, criar o primeiro painel.

CÓPIA DO ALUNO

Tudo que for visto em teoria será reproduzido na demonstração ao vivo. Em seguida, você repete no seu ambiente. O objetivo é que, ao final da noite, o Grafana esteja rodando na sua máquina e você saiba se virar sozinho.

Material do aluno — uso pessoal, sem redistribuição

CONCEITO

O que é observabilidade

Observabilidade é a prática de compreender o estado interno de um sistema a partir dos dados que ele emite. Vai além do monitoramento tradicional: a pergunta deixa de ser 'está no ar?' e passa a ser 'por que está se comportando assim?'.

Três pilares sustentam essa prática. Métricas — valores numéricos agregados no tempo, leves de coletar e ideais para tendências. Logs — registros textuais de eventos, ricos em contexto, usados para investigar incidentes. Traces — o caminho de uma requisição entre serviços, fundamental em arquiteturas distribuídas.

Definição formal

'Observabilidade é a medida de quão bem você pode inferir o estado interno de um sistema a partir do conhecimento de suas saídas externas.'

— Norbert Wiener, 1960

Aplicada a TI: quanto mais ricos e bem correlacionados forem os sinais emitidos por um sistema, mais fácil será diagnosticar problemas sem precisar entrar nele.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Métricas, logs e traces

Métricas

Números ao longo do tempo: CPU, latência, requisições por segundo. Caras: armazenamento barato, alertas simples. Mãe: séries temporais em bancos como Prometheus, VictoriaMetrics, Mimir.

Logs

Eventos textuais emitidos pela aplicação. Caros em volume mas riquíssimos em contexto. Stack típica: Promtail/Alloy + Loki ou Filebeat + Elasticsearch.

Traces

Caminho de uma requisição por múltiplos serviços. Fundamentais em microsserviços. OpenTelemetry é o padrão de instrumentação; Tempo e Jaeger armazenam.

Grafana é a camada visual: consome os três pilares e os correlaciona em um único painel.

CÓPIA DO ALUNO

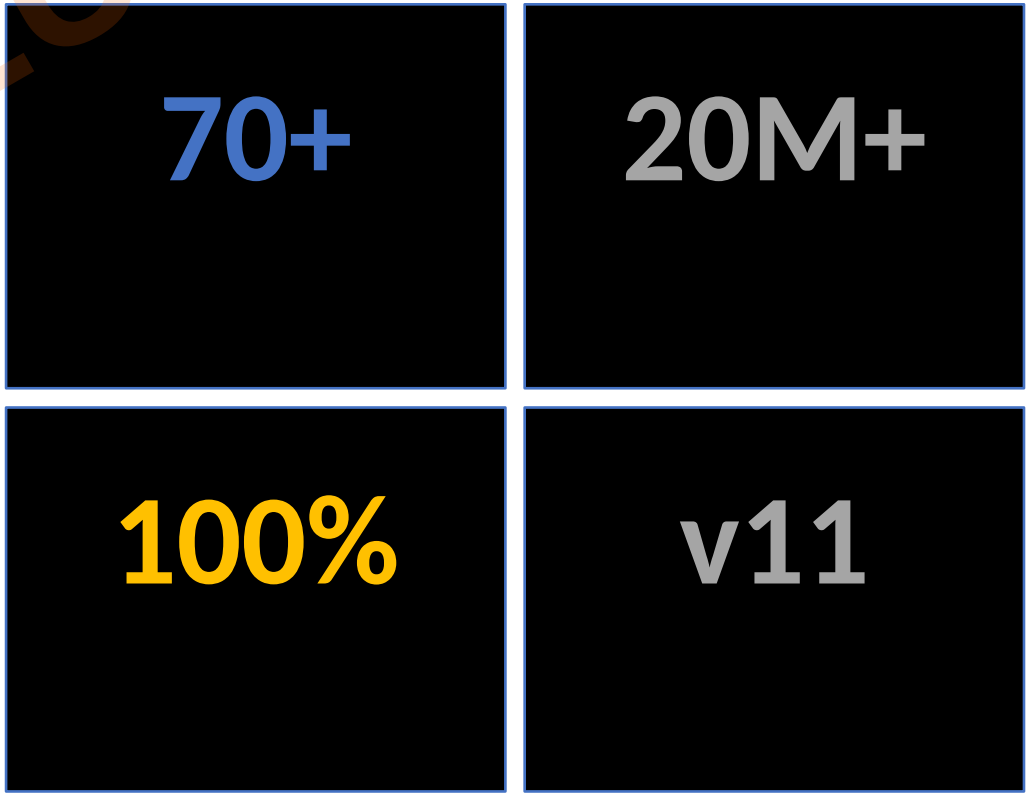
Material do aluno — uso pessoal, sem redistribuição

O que é o Grafana

Grafana é uma plataforma open-source de visualização e análise de dados, criada em 2014 pela Grafana Labs. Originalmente focada em séries temporais, hoje é o ponto de convergência para métricas, logs e traces — desde bancos como Prometheus até serviços SaaS, REST APIs e arquivos CSV.

A versão atual é a 11.x, com interface renovada, novo sistema de painéis e suporte oficial a data sources como Prometheus, Loki, Tempo, Elasticsearch, InfluxDB, MySQL, PostgreSQL, BigQuery, Datadog, CloudWatch e mais de 70 outros.

A licença é AGPLv3 para a edição open-source. A Grafana Labs oferece também versões gerenciadas (Grafana Cloud) e Enterprise.



CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Como o Grafana se conecta ao mundo

1 Data source

Prometheus, Loki, MySQL, CloudWatch — qualquer backend com driver.

2 Query

Cada fonte tem sua linguagem: PromQL, LogQL, SQL. Grafana entende todas.

3 Transformações

Filter, join, calculate, regex. Manipula dados antes de plotar.

4 Painel

Time series, stat, gauge, bar, table, heatmap, geomap, candlestick.

5 Dashboard

Coleção de painéis com variáveis, anotações e links cruzados.

Esse fluxo é o que você verá em ação na demonstração. Vamos criar um dashboard que consome métricas do Prometheus, transforma com regex e mostra três tipos de painel lado a lado.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Benefícios da observabilidade com Grafana

- Unifica dados de múltiplas fontes em um único painel
- Reduz o tempo médio de diagnóstico (MTTR) de incidentes
- Open source, sem custos de licenciamento por usuário ou painel
- Comunidade ativa com 10.000+ dashboards prontos para importar
- Provisionamento via arquivo (GitOps) para ambientes de produção
- Alertas nativos com roteamento por canais (e-mail, Slack, webhook)
- API HTTP completa para automação e integração com outras ferramentas

Em números (Grafana Labs 2024)

- **20M+ usuários ativos mensais**
- **1.000+ empresas pagantes (Grafana Cloud + Enterprise)**
- **100k estrelas no GitHub**
- **4.000+ contribuições da comunidade por ano**

É a ferramenta de visualização mais usada no mercado de DevOps, segundo surveys da CNCF.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Séries temporais — o tipo de dado mais comum

Grande parte dos dados que você vai visualizar no Grafana são séries temporais: sequências de medições indexadas por timestamp.

Cada ponto tem três componentes:

- Métrica: o nome do que está sendo medido (node_cpu_seconds_total)
- Timestamp: o instante da coleta (2024-11-20T18:30:00Z)
- Valor: o número em si (12.5)
- Labels: dimensões que qualificam a métrica (instance=host-01, mode=idle)

Os labels são o que tornam o Prometheus e o Grafana tão poderosos: você consegue agregar, filtrar e cruzar métricas por qualquer dimensão, sem mover dados de lugar.

```
# Exemplo de série temporal

node_cpu_seconds_total{
  instance="host-01",
  mode="idle",
  cpu="0"
} 18423.45 @ 1732128600

node_cpu_seconds_total{
  instance="host-01",
  mode="idle",
  cpu="1"
} 18425.12 @ 1732128600

# Mesmo nome de métrica,
# labels diferentes = séries
# diferentes. O Grafana plota
# cada série como uma linha.
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

DEMONSTRAÇÃO

Subindo o ambiente — visão geral

Para a primeira noite, vamos rodar 3 containers Docker lado a lado:

1. Prometheus (porta 9090) — banco de séries temporais que raspa métricas dos exporters
2. Node Exporter (porta 9100) — emite métricas do host (CPU, memória, disco, rede)
3. Grafana (porta 3000) — interface visual que consome o Prometheus

Tudo é provisionado automaticamente: o Prometheus já tem o Node Exporter como target, e o Grafana já tem o Prometheus como datasource. Sem cliques manuais.

```
# Estrutura do lab

lab/dia1/
├── docker-compose.yml
├── prometheus.yml
├── alerts.yml
└── grafana/
    ├── provisioning/
    │   ├── datasources/
    │   │   └── datasource.yml
    │   └── dashboards/
    │       └── dashboards.yml
    └── dashboards/
        └── 01-visao-geral-host.json
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

DEMO

docker compose up — em 1 comando

O aluno só precisa executar dois comandos:

1. cd lab/dia1
2. docker compose up -d

Em ~30 segundos, os 3 containers estão em pé, o Prometheus já está raspando o Node Exporter a cada 15 segundos, e o Grafana carrega o dashboard pré-configurado.

Toda a complexidade está isolada nos arquivos de configuração que o instrutor já preparou. O aluno não digita nada complicado — só observa e entende o que cada arquivo faz.

```
$ cd lab/dia1
$ docker compose up -d
[+] Running 3/3
 ✔ Container lab-prometheus    Started
 ✔ Container lab-node-exporter Started
 ✔ Container lab-grafana       Started

$ docker compose ps
NAME                STATUS    PORTS
lab-prometheus      Up        0.0.0.0:9090->9090/tcp
lab-node-exporter   Up        0.0.0.0:9100->9100/tcp
lab-grafana         Up        0.0.0.0:3000->3000/tcp
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

HANDS-ON

Exercício 1.1 — Subir o ambiente

Você tem 5 minutos. Abra o terminal e siga exatamente os passos abaixo.

```
cd lab/dia1
docker compose up -d
docker compose ps

# Aguarde ~30 segundos e abra o navegador:
# Grafana:    http://localhost:3000   (admin / admin)
# Prometheus: http://localhost:9090
# Métricas:   http://localhost:9100/metrics
```

✅ Critério de aceite: três containers em status Up, Grafana abrindo no navegador, login admin / admin funcionando.

⚠ Se algum container ficar em Restarting, rode: docker compose logs -f <nome> e chame o instrutor.

CÓPIA DO ALUNO

Conhecendo a interface do Grafana

A barra lateral esquerda é o ponto de entrada para todos os recursos:

- Search (lupa) — busca por painéis, dashboards e data sources
- Dashboards — biblioteca de dashboards (4 quadradinhos)
- Explore — modo ad-hoc para testar queries sem criar dashboard
- Alerting — regras, silences, contact points e grupos
- Connections — gerenciamento de data sources e plugins
- Administration — usuários, times, organizações, configurações

O topo da tela tem o seletor de tempo (right side) e o botão de refresh. Esses dois controles são os mais usados no dia a dia — vale a pena memorizar.

Atalhos úteis

G + D → ir para Dashboards
G + E → ir para Explore
Ctrl+K → command palette (busca global)
Esc → fecha menus
? → lista todos os atalhos

Na dúvida, Ctrl+K é seu melhor amigo. O command palette indexa todos os recursos do Grafana.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Explore — o laboratório de queries

O Explore é onde você testa queries antes de colocá-las em um dashboard. É a ferramenta de investigação — equivalente ao psql para o PostgreSQL, ou ao curl para uma API.

No Dia 1 vamos usar quase exclusivamente. Selecione o datasource Prometheus, digite a query, escolha o intervalo de tempo e clique em Run query.

Três botões ficam disponíveis acima dos resultados: Table (mostra o JSON cru), Time series (gráfico de linha) e Inspector (metadados, tempo de execução, status da query).

```
# Suas primeiras queries no Explore

# 1. Valor bruto
node_cpu_seconds_total

# 2. Taxa por segundo
rate(node_cpu_seconds_total[1m])

# 3. CPU idle médio
avg(rate(node_cpu_seconds_total{mode="idle"}
[1m]))

# 4. CPU em uso (%)
100 -
(avg(rate(node_cpu_seconds_total{mode="idle"}
[1m])) * 100)

# Dica: Ctrl+Space mostra auto-complete
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

CONCEITO

Linguagens de consulta por data source

Data source	Linguagem	Caso de uso
Prometheus	PromQL	Métricas de infraestrutura, microsserviços, Kubernetes
Loki	LogQL	Logs estruturados, debugging, auditoria
Tempo / Jaeger	TraceQL	Tracing distribuído, latência por hop
MySQL / PostgreSQL	SQL	Métricas de negócio, dashboards de aplicação
InfluxDB / VictoriaMetrics	Flux / PromQL-like	Séries temporais de IoT, edge, telemetria
CloudWatch / Azure Monitor	Linguagem própria	Métricas de provedores cloud

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

HANDS-ON

Exercício 1.2 — Explorar dados no Explore

```
# Passo a passo

1. Menu lateral → Explore
2. Data source: Prometheus
3. Cole a query abaixo e clique em Run query:

   rate(node_cpu_seconds_total{mode="idle"}[1m])

4. Mude o intervalo de tempo para "Last 5 minutes"
5. No canto direito do gráfico, clique em "Table" — veja o JSON cru
6. Volte para "Time series"
7. Agora troque a query para:

   100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)

8. Adicione um unit "Percent (0-100)" em Panel options (mesmo sem salvar)

Pergunta: o que mudou entre a query 1 e a 3? Anote sua resposta no caderno.
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Criando seu primeiro dashboard

A criação de um dashboard segue um fluxo direto:

1. Dashboards → New → New dashboard
2. Add visualization (ou Ctrl+V dentro do dashboard)
3. Escolha o data source (já pré-selecionado se for o default)
4. Escreva a query no editor PromQL
5. Configure título, unidade, thresholds, cores
6. Apply → o painel aparece no dashboard
7. Salve com Ctrl+S

Você pode arrastar painéis para reordenar, redimensionar pelas bordas e agrupar com 'Rows' para organização lógica. Tudo é não-destrutivo: pode editar e reverter a qualquer momento.

Dica de ouro

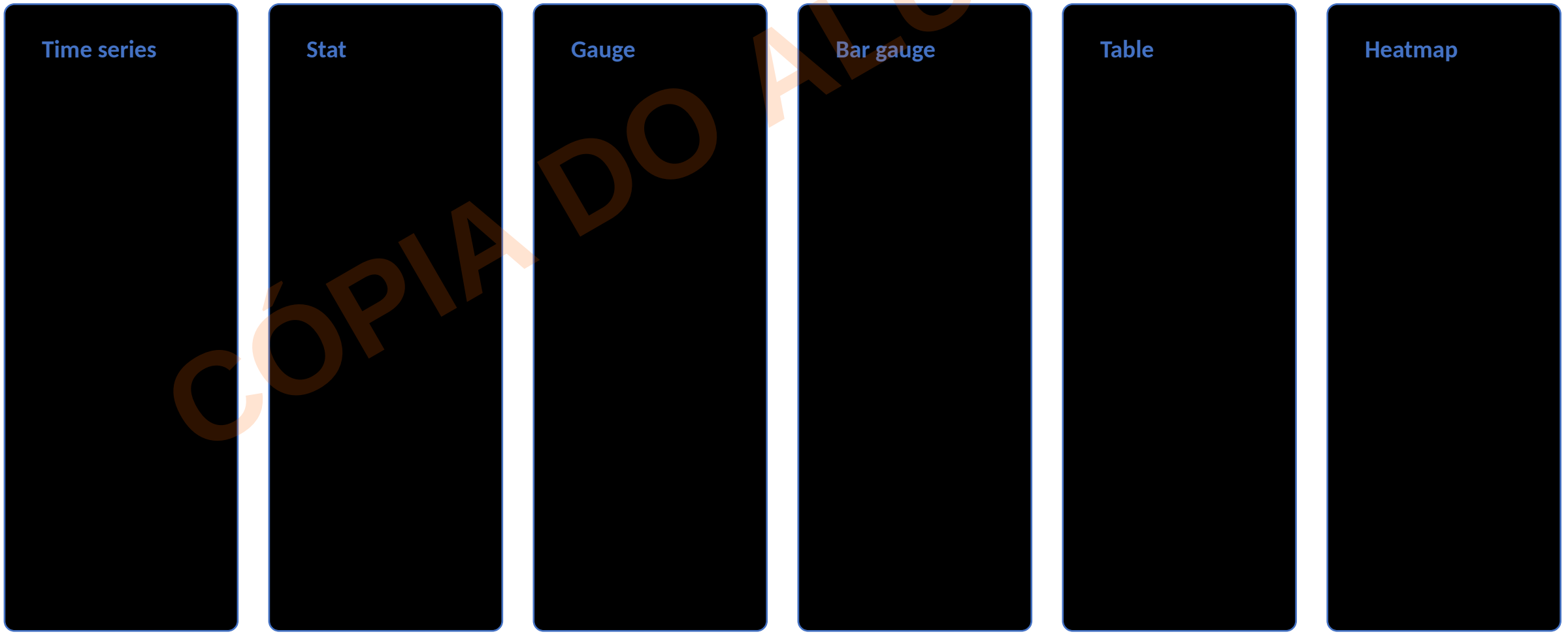
Não crie o dashboard definitivo logo de cara. Crie um draft, ajuste a query no Explore até ficar perfeita, copie a query para o painel, e só então refine o visual.

O Explore é o sandbox. O dashboard é o produto final.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Tipos de visualização mais usados



CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

HANDS-ON

Exercício 1.3 — Dashboard do zero

Crie um dashboard chamado 'Meu primeiro dashboard' com 2 painéis:

Painel 1 — Stat: Uso de CPU do host (%)

- Type: stat
- Query: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
- Unit: Percent (0-100)
- Thresholds: 0–60 verde, 60–80 amarelo, 80+ vermelho

Painel 2 — Time series: Memória em uso (bytes)

- Type: time series
- Query: node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes
- Unit: bytes
- Fill opacity: 20
- Line width: 2

```
# Estrutura final do dashboard

┌──────────────────────────────────────────┐
│  Meu primeiro dashboard                  │
├──────────────────┬───────────────────────┤
│   Stat           │    Time series        │
│                  │                       │
│   12%            │   /\_                 │
│   CPU em uso     │  /    \_              │
│                  │ /       \_            │
│                  │/          \_          │
└──────────────────┴───────────────────────┘

Salvar: Ctrl+S
Compartilhar: Share → Export
```

CÓPIA DO ALUNO

Recapitulando o Dia 1

- Observabilidade é a disciplina de entender o sistema a partir das suas saídas
- Os 3 pilares: métricas, logs e traces — Grafana visualiza os três
- Prometheus é o banco de séries temporais mais usado no mercado
- Node Exporter emite métricas do host em formato Prometheus
- Subir a stack leva 30 segundos com Docker Compose
- A interface do Grafana é organizada em torno de dashboards e Explore
- PromQL é a linguagem para extrair e transformar séries temporais
- Os tipos de painel mais comuns: time series, stat, gauge, table, heatmap
- O ciclo de criação: testa no Explore → cria painel → ajusta → salva

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

DIA 02

Visualizações, variáveis e alertas

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Grafana: Visualizando Dados

Aprofundando no Grafana

1 **Variáveis**

Dashboards dinâmicos com dropdowns de host, região, intervalo.

2 **Mais tipos**

Gauge, table, bar gauge, heatmap em ação.

3 **PromQL**

topk, agregação por label, funções de janela.

4 **Alertas**

Configurar regras Prometheus e Grafana, ver FIRING.

5 **Containers**

cAdvisor para métricas Docker no dashboard.

CÓPIA DO ALUNO

Hoje o ambiente ganha mais componentes: Alertmanager e cAdvisor. O objetivo é construir dashboards profissionais — com variáveis, filtros e alertas que disparam de verdade.

Material do aluno — uso pessoal, sem redistribuição

CONCEITO

Variáveis — dashboards que se adaptam

Variáveis transformam um dashboard estático em uma ferramenta dinâmica. Em vez de criar um painel por host, você cria um painel com uma variável $host e o usuário escolhe no dropdown.

Tipos principais:

- Query — valores vêm de uma query (ex: hosts do Prometheus)
- Custom — lista fixa definida por você (ex: ambiente = prod,staging,dev)
- Data source — alterna entre múltiplos data sources
- Interval — intervalos de tempo pré-definidos (5m, 1h, 1d)
- Text box — texto livre do usuário

```
# Criando uma variável no dashboard

Settings (engrenagem) → Variables → New variable

Name:        host
Type:        Query
Data source: Prometheus
Query:       label_values(node_cpu_seconds_total,
instance)
Refresh:     On time range change
Multi-value: true   (selecionar vários)
Include All: true   (opção 'All' no topo)

# Uso nas queries
node_cpu_seconds_total{instance=~"$host"}
node_memory_MemTotal_bytes{instance=~"$host"}
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

HANDS-ON

Exercício 2.1 — Variável $host

No dashboard '02 - Host dinâmico (variáveis)':

1. No topo, mude o dropdown 'host' para um único host e observe todos os painéis se atualizarem
2. Mude para 'All' — todos os painéis passam a mostrar múltiplas séries
3. Adicione uma variável nova: tipo Custom, nome cpu_mode, valores idle,user,system,iowait
4. Em uma query existente, troque mode="idle" por mode=~"$cpu_mode"
5. Salve o dashboard como '02 - Host dinâmico (variáveis) — meu'

Dica: o regex =~ aceita múltiplos valores separados por pipe. mode=~"idle|user" casa com mode=idle OU mode=user.

```
# Query original (fixa)
100 - (avg by(instance) (
  rate(node_cpu_seconds_total{mode="idle"}[5m])
) * 100)

# Query com variável
100 - (avg by(instance) (
  rate(node_cpu_seconds_total{
    mode=~"$cpu_mode",
    instance=~"$host"
  }[5m])
) * 100)

# $__rate_interval é uma variável
# especial que se adapta ao range
# de tempo escolhido.
```

CÓPIA DO ALUNO

PAINEL

Gauge — quando o valor importa mais que a série

O gauge mostra um valor único em relação a um limite. É o velocímetro do painel — útil quando o que importa é 'estamos em qual faixa?'.

Exemplos de uso apropriado:

- Memória em uso (0-100%)
- Conexões ativas vs limite da pool
- Tamanho de fila vs SLO
- SLA de uptime mensal

Não use para séries temporais. Para isso, use time series. Gauge sem tendência temporal vira ruído visual.

```
# Configuração recomendada

Type:   gauge
Unit:   Percent (0-100)
Min:    0
Max:    100
Thresholds:
  0  – 70  → green
  70 – 85  → orange
  85 – 100 → red

# Query típica
(1 - (node_memory_MemAvailable_bytes
       / node_memory_MemTotal_bytes)) * 100
```

CÓPIA DO ALUNO

Heatmap — padrões temporais em um piscar

Heatmap mapeia duas dimensões em uma grade de cores: tempo no eixo X, valor no eixo Y, intensidade da cor indicando frequência.

É especialmente útil para identificar padrões cíclicos — picos de tráfego que se repetem toda terça-feira, lentidão à meia-noite, jobs pesados em horário comercial.

A query precisa retornar o formato heatmap: agregações em buckets. O Grafana faz isso automaticamente se você marcar 'Format as Heatmap' na query.

```
# Query para heatmap (formato heatmap)

100 - (avg by(instance) (
  rate(
    node_cpu_seconds_total{
      mode="idle",
      instance=~"$host"
    }[5m]
  )
) * 100)

# Na query do Prometheus:
# - Marque 'Format: Heatmap'
# - Aumente o range para 6h+ para
#   ver padrões
# - Y-axis: linear ou log
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

topk e agregação por label

Duas funções do PromQL mudam completamente o que você vê:

- topk(N, expr) — retorna os N maiores valores de uma expressão
- sum by(label) (expr) — soma os valores agrupando por label
- avg by(label) (expr) — calcula média por grupo
- count by(label) (expr) — conta séries distintas
- rate / irate — taxa de variação por segundo

Combinando essas funções, você constrói queries como 'top 5 hosts com mais CPU' ou 'uso médio por datacenter'.

```
# Top 5 containers por CPU
topk(5, rate(
  container_cpu_usage_seconds_total{name!=""}[5m]
) * 100)

# Memória total por datacenter
sum by(datacenter) (
  node_memory_MemTotal_bytes
  - node_memory_MemAvailable_bytes
)

# Contagem de hosts UP
count(up{job="node-exporter"} == 1)

# Taxa de crescimento por minuto
rate(node_network_receive_bytes_total[5m]) * 60
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Alertas no Prometheus — anatomia

Uma regra de alerta tem 4 partes:

1. A query que produz o sinal (expr)
2. A condição de disparo (> 80, == 0, etc.)
3. O tempo de espera (for: 5m) — evita alertas espúrios
4. As anotações — summary e description que aparecem na notificação

O alerta fica em 3 estados: Inactive (sem disparar), Pending (condição atingida, aguardando o for), Firing (passou o for, está notificando).

Acompanhe em http://localhost:9090/alerts.

```
groups:
  - name: host
    rules:
      - alert: HostCPUAlta
        expr: |
          100 - (avg by(instance) (
            rate(node_cpu_seconds_total{
              mode="idle"
            }[5m])
          ) * 100) > 80
        for: 5m
        labels:
          severidade: warning
        annotations:
          summary: "CPU alta em
{{ $labels.instance }}"
          description: |
            CPU acima de 80% por mais de
            5 minutos (atual: {{ $value | printf
\"%.1f\" }}%)
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

HANDS-ON

Exercício 2.2 — Disparar um alerta

```
# Em outro terminal, force carga no container
docker exec -it lab-node-exporter sh -c \
  'dd if=/dev/urandom of=/dev/null bs=1M count=2000 &'

# Acompanhe o alerta em http://localhost:9090/alerts
# Status esperado: PENDING → FIRING (após 5 min)

# Quando quiser parar, mate o processo:
docker exec -it lab-node-exporter pkill dd

# O alerta volta a INACTIVE após 5 min
# de CPU abaixo do threshold.
```

Por que isso é importante: o aluno vê o ciclo de vida real do alerta (Pending → Firing → Inactive), entende o papel do 'for' e por que a notificação não é instantânea. Em produção, isso evita alertas de ruído.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Alertas no Grafana — UI vs arquivo

O Grafana tem seu próprio motor de alertas, separado do Prometheus. Você pode usar qualquer um — ou os dois.

UI (Alerting → Alert rules):

- Bom para prototipar
- Editor visual com preview
- Cria contact points e silences

Arquivo (provisioning/alerting/):

- GitOps-friendly
- Versionado junto com a infra
- Idempotente: aplica a cada reload

A configuração file-based é o padrão em produção. Vamos exercitar os dois.

```
# Grafana - alert rule via arquivo YAML

apiVersion: 1
groups:
  - orgId: 1
    name: regras_padrao
    folder: Laboratório
    interval: 1m
    rules:
      - uid: cpu-alta-grafana
        title: CPU alta no host
        condition: A
        data:
          - refId: A
            datasourceUid: prometheus
            model:
              expr: |
                100 - (avg by(instance) (
                  rate(node_cpu_seconds_total{
                    mode="idle"
                  }[5m])
                ) * 100)
        for: 5m
        labels:
          severidade: warning
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

cAdvisor — métricas de container

cAdvisor é o agente oficial do Kubernetes para métricas de container. Roda como container e expõe, na porta 8080, métricas Prometheus sobre cada container do host.

A métrica principal é container_cpu_usage_seconds_total — uso de CPU por container em segundos por segundo. Combine com topk para ranking.

Use o dashboard '03 - Containers (cAdvisor)' para ver a stack atual.

```
# Top 5 containers por CPU
topk(5, rate(
  container_cpu_usage_seconds_total{name!=""}[5m]
) * 100)

# Memória por container
topk(5, container_memory_usage_bytes{name!=""})

# I/O de disco por container
rate(container_fs_reads_bytes_total[5m])

# Rede por container
rate(container_network_receive_bytes_total[5m])
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

RECAP

Recapitulando o Dia 2

- Variáveis transformam dashboards em ferramentas dinâmicas
- Tipos de variável: query, custom, data source, interval, text box
- Gauge para valor único em relação a limite; heatmap para padrões temporais
- topk e agregação por label são funções-chave do PromQL
- rate() converte contador em taxa por segundo; irate() é a versão instantânea
- Alertas têm 3 estados: Inactive, Pending, Firing — o 'for' evita ruído
- Grafana tem motor próprio de alertas, separado do Prometheus
- cAdvisor expõe métricas de container no formato Prometheus
- Provisionamento file-based (YAML) é o padrão em produção

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

DIA 03

Instalação no Ubuntu (prática guiada)

CÓPIA DO ALUNO

Grafana: Visualizando Dados

Instalação completa do zero

1 — Preparar

Sistema, repositório, usuário, firewall.

2 — Instalar

Grafana 11.3 via APT oficial, Prometheus e Node Exporter.

3 — Configurar

grafana.ini, datasource provisionado, systemd.

4 — Validar

Serviços ativos, portas abertas, targets UP.

5 — Customizar

Trocar porta, configurar SMTP, explorar logs.

CÓPIA DO ALUNO

O instrutor projeta a tela e executa junto. Os scripts são comentados e idempotentes — se algo der errado, basta rodar de novo. Os scripts estão em lab/dia3/.

Material do aluno — uso pessoal, sem redistribuição

CONCEITO

Por que instalar do zero se o Docker resolve?

Docker é ótimo para laboratório, mas em produção a maioria das empresas precisa instalar binários diretamente na máquina virtual — sem camada de container entre.

Ao instalar do zero, você entende:

- Onde o Grafana guarda configuração (/etc/grafana)
- Onde ficam os logs (/var/log/grafana)
- Como o systemd gerencia o serviço
- Quais portas e usuários o processo cria
- Como atualizar com apt upgrade

Esse conhecimento é a base para troubleshooting real.

Regra de bolso

Use Docker para dev/teste. Use pacote do sistema para produção (exceto Kubernetes, onde container é a regra).

Em ambos os casos, o Grafana é configurado por arquivos em /etc/grafana. O comportamento é idêntico.

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

01-prepare.sh — preparando o sistema

O primeiro script faz o trabalho de base:

1. apt-get update — sincroniza o índice de pacotes
2. apt-get install de pré-requisitos (curl, gnupg, apt-transport-https)
3. Criação do usuário grafana — sem login, sem senha
4. Criação dos diretórios /var/lib/grafana, /var/log/grafana, /etc/grafana
5. Abertura das portas 3000 e 9090 no UFW, se ativo

O set -euo pipefail garante que o script pare no primeiro erro.

```
#!/usr/bin/env bash
set -euo pipefail

if [[ $EUID -ne 0 ]]; then
  echo "ERRO: execute como root"
  exit 1
fi

apt-get update -y
apt-get install -y \
  curl \
  apt-transport-https \
  software-properties-common \
  gnupg \
  lsb-release \
  ca-certificates

# Cria usuário sem login
adduser --system \
  --no-create-home \
  --shell /bin/false \
  --group grafana

# Diretórios com dono correto
install -d -o grafana -g grafana \
  /var/lib/grafana /var/log/grafana
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

02-install-grafana.sh — via repositório APT

Por que adicionar um repositório em vez de baixar o .deb?

- apt entende dependências automaticamente
- apt upgrade atualiza o Grafana junto com o sistema
- A chave GPG garante autenticidade do pacote

A versão fica fixada (11.3.0) para reprodutibilidade. Em produção, pode-se usar grafana (latest) ou travar com versionamento semver.

```
#!/usr/bin/env bash
set -euo pipefail

# Chave GPG oficial
curl -fsSL https://apt.grafana.com/gpg.key \
  | gpg --dearmor \
    -o /etc/apt/keyrings/grafana.gpg

# Entrada do repositório
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] \
https://apt.grafana.com stable main" \
  > /etc/apt/sources.list.d/grafana.list

apt-get update -y
apt-get install -y \
  --no-install-recommends \
  grafana=11.3.0

systemctl daemon-reload
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

03-configure.sh — grafana.ini + provisionamento

O grafana.ini tem mais de 1000 linhas comentadas. Em vez de editá-lo à mão, usamos sed para ajustar apenas o que precisamos:

- http_port = 3000
- admin_user / admin_password
- database path (SQLite local)
- provisioning path

Em seguida provisionamos o datasource do Prometheus via YAML — assim ele sobe automaticamente junto com o Grafana.

```
# Ajustando o grafana.ini

sed -i 's/^;http_port = 3000/http_port = 3000/' \
  /etc/grafana/grafana.ini

sed -i 's/^;admin_user = admin/admin_user = admin/' \
  /etc/grafana/grafana.ini

sed -i 's/^;admin_password = admin/admin_password =
admin/' \
  /etc/grafana/grafana.ini

# Provisiona Prometheus
cat > /etc/grafana/provisioning/datasources/\
prometheus.yml <<'YAML'
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    url: http://localhost:9090
    isDefault: true
YAML

# Habilita e inicia
systemctl enable --now grafana-server
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

04-install-prometheus.sh — binário oficial

O Prometheus é distribuído como binário tar.gz. Instalamos em /usr/local/bin (padrão de binários não-APT) e criamos o serviço systemd do zero.

A flag --web.enable-lifecycle é importante: ela expõe o endpoint POST /-/reload, que recarrega a configuração sem reiniciar o processo. Usado em produção para aplicar mudanças sem downtime.

```
# Service unit
[Unit]
Description=Prometheus
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus \
  --web.enable-lifecycle
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

05-install-node-exporter.sh

Node Exporter é simples: baixa, instala em /usr/local/bin, cria unit systemd. Não tem banco — apenas lê /proc e /sys e expõe via HTTP.

Por padrão, escuta em todas as interfaces na porta 9100. Em produção, recomenda-se escutar só em localhost e deixar o Prometheus raspar localmente (reduz superfície de ataque).

```
[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter
Restart=always

# Para escutar só em localhost:
# ExecStart=/usr/local/bin/node_exporter \
#   --web.listen-address=127.0.0.1:9100
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

06-validate.sh — check-list automatizado

O último script verifica 6 coisas:

1. Serviços systemd ativos (grafana-server, prometheus, node_exporter)
2. Portas em escuta (3000, 9090, 9100)
3. Health-check HTTP (curl em /api/health, /-/healthy, /metrics)
4. Targets Prometheus (todos UP?)
5. DataSources do Grafana (listados?)
6. Mensagem final com URLs e credenciais

Cada check imprime [OK] verde ou [FAIL] vermelho e aborta em caso de erro. O aluno vê, em segundos, o que está ou não funcionando.

```
== Targets do Prometheus ==
  job=prometheus        health=up
  job=node              health=up

== Datasources do Grafana ==
  Prometheus  type=prometheus
              url=http://localhost:9090

Tudo certo. Acesse:
  Grafana:    http://localhost:3000
  Prometheus: http://localhost:9090
  Métricas:   http://localhost:9100/metrics
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

TROUBLESHOOTING

Problemas comuns e como resolver

Sintoma	Causa provável	Solução
Port 3000 in use	Outro processo usando a porta	sudo lsof -i :3000 → kill ou trocar a porta
Permission denied em /var/lib/grafana	Dono do diretório errado	sudo chown -R grafana:grafana /var/lib/grafana
Prometheus target DOWN	Node Exporter não está respondendo	systemctl status node_exporter, curl localhost:9100
Datasource not found no Grafana	Provisionamento não aplicou	rm -rf /var/lib/grafana/grafana.db (força reload)
Login admin não funciona	Senha não foi definida	Editar /etc/grafana/grafana.ini → admin_password
apt-get update falha	Repositório sem chave ou off-line	Verificar /etc/apt/keyrings/grafana.gpg

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

HANDS-ON

Exercício 3.1 — Execução completa

```
# Sequência — execute em ordem
sudo bash 01-prepare.sh
sudo bash 02-install-grafana.sh
sudo bash 03-configure.sh
sudo bash 04-install-prometheus.sh
sudo bash 05-install-node-exporter.sh
sudo bash 06-validate.sh
```

Antes de começar: confirme com o instrutor que sua VM Ubuntu tem pelo menos 2 vCPU e 4 GB de RAM.

Responda no caderno: qual é o nome do usuário criado para o Grafana, em qual porta ele escuta, e onde fica o banco SQLite?

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

O que aprendemos hoje

- Repositório APT oficial é a forma canônica de instalar o Grafana em produção
- Chave GPG em /etc/apt/keyrings garante autenticidade do pacote
- Fixar versão (grafana=11.3.0) traz reprodutibilidade ao ambiente
- systemd gerencia o serviço — enable + start para subir no boot
- Provisionamento file-based deixa configuração versionada no Git
- Cada serviço roda com usuário próprio — princípio do menor privilégio
- O diretório /var/lib/grafana guarda o SQLite e os plugins
- O diretório /etc/grafana tem configuração e provisionamento
- Logs ficam em /var/log/grafana/grafana.log
- Validação automatizada economiza horas de troubleshooting

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

DIA 04

Temas, provisionamento & multi-stack

CÓPIA DO ALUNO

Encerramento com produção real

1. **Plugins** — Instalar e gerenciar plugins da comunidade e oficiais.
2. **Theming** — Dark mode, customização por usuário e por organização.
3. **GitOps** — Provisionamento file-based de dashboards, alerts, datasources.
4. **Multi-stack** — Adicionar Loki para logs e Tempo para traces.
5. **Encerramento** — Q&A, próximos passos, certificação interna.

CÓPIA DO ALUNO

Hoje fechamos o ciclo: o aluno sai capaz de operar Grafana em produção, com versionamento no Git, dashboards reproduzíveis e logs/traces integrados.

Material do aluno — uso pessoal, sem redistribuição

Ecossistema de plugins

O Grafana tem três tipos de plugins:

- Panels — novos tipos de visualização (piechart, worldmap, flowchart)
- Data sources — novos conectores (Splunk, Datadog, Stripe, Salesforce)
- Apps — bundles que incluem painéis, datasources e dashboards prontos

Instalação: grafana-cli plugins install <id>. Reinício obrigatório.

São armazenados em /var/lib/grafana/plugins. Para desabilitar: remover o diretório e reiniciar.

```
# Instalando um plugin
docker exec -it lab-grafana bash
  grafana-cli plugins install grafana-piechart-
panel
  exit
docker compose restart grafana

# Plugins populares
• grafana-piechart-panel   (gráfico de pizza)
• grafana-worldmap-panel    (mapa geográfico)
• grafana-clock-panel       (relógio/calendário)
• grafana-polystat-panel    (múltiplas stats)
• grafana-googlesheets-datasource
• marcusolsson-json-datasource

# Listar instalados
grafana-cli plugins ls
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Tema visual — por usuário e por dashboard

O Grafana suporta múltiplos temas visuais que vão do light ao ultra-dark. O usuário pode escolher o seu em Preferences.

Para personalizar por dashboard, use o JSON model: o campo 'style' aceita 'dark' ou 'default'. Em produção, prefira 'dark' para painéis em NOC e 'default' para apresentações executivas.

Cores dos painéis: cada série aceita um override no campo 'color' do Field config. Use paleta consistente entre dashboards da mesma organização.

```
# Trocar o tema de um dashboard por JSON

{
  "title": "Meu dashboard",
  "style": "dark",
  "panels": [
    {
      "fieldConfig": {
        "defaults": {
          "color": {
            "mode": "palette-classic"
          }
        }
      }
    }
  ]
}

# Forçar tema em URL
?theme=dark
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Importar dashboards prontos

Grafana.com hospeda milhares de dashboards oficiais e da comunidade. Os mais populares:

- 1860 — Node Exporter Full (esse é o clássico)
- 315 — Docker Container & Host Metrics
- 6417 — Kubernetes Cluster Monitoring
- 3662 — Prometheus 2.0 Stats
- 12708 — Loki Stack

A importação pede apenas o data source. Grafana reescreve os UIDs automaticamente.

```
# Como importar

1. Dashboards → New → Import
2. Opção A: digite o ID (1860)
3. Opção B: cole a URL
(https://grafana.com/grafana/dashboards/1860)
4. Opção C: faça upload de um .json
5. Escolha o data source de destino
6. Clique em Import

# Exportar o seu
Share → Export → Save to file

# Importar por API
curl -X POST \
  -H 'Content-Type: application/json' \
  -d @dashboard.json \
  -u admin:admin \
  http://localhost:3000/api/dashboards/import
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Provisionamento file-based

Em produção, clicar no Grafana UI para criar dashboards é o primeiro passo para o caos. Use file-based provisioning: arquivos YAML/JSON em uma pasta, versionados no Git.

Vantagens:

- Reprodutibilidade: o mesmo arquivo gera o mesmo ambiente
- Auditoria: git log mostra quem mudou o quê e quando
- Disaster recovery: rm grafana.db + reload restaura tudo
- CI/CD: deploy automático junto com a infra

Estrutura típica:

```
/etc/grafana/provisioning/
├── datasources/
│   └── prometheus.yml
├── dashboards/
│   └── provider.yml
└── alerting/
    └── rules.yml

/var/lib/grafana/dashboards/
├── 01-visao-geral.json
├── 02-host-dinamico.json
└── 03-containers.json

# Os YAMLs definem 'onde estão os
# recursos'; os JSONs/recursos
# ficam ao lado ou em volume
# montado.
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Adicionando Loki — logs no Grafana

Loki é o 'Prometheus dos logs' — coleta, indexa e armazena logs de aplicações. Foi criado pela Grafana Labs com a mesma filosofia do Prometheus: baixo custo, fácil de operar, integração nativa.

A query language é a LogQL, similar ao PromQL mas com operadores para filtrar texto ({app="api"} |= "error").

Para coletar logs dos containers, use Promtail (ou Grafana Alloy, o substituto). Em Kubernetes, ele lê direto do kubelet.

```
# Adicionar Loki ao docker-compose

services:
  loki:
    image: grafana/loki:3.3.2
    ports: ["3100:3100"]
    command: -config.file=/etc/loki/local-
config.yaml
    networks: [lab-net]

# Adicionar datasource no Grafana
# Connections → Data sources → Add
# Type: Loki
# URL:  http://loki:3100

# Query de exemplo no Explore
{app="nginx"} |= "error" | json | line_format
"{{.message}}"
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Adicionando Tempo — traces distribuídos

Tempo é o backend de traces open-source da Grafana Labs. Substitui o Jaeger em muitos casos, com integração nativa ao Grafana.

A query language é a TraceQL — também similar ao PromQL.

Com Loki + Tempo + Prometheus no mesmo dashboard, o SRE pula de uma métrica para o trace relacionado, e do trace para o log de um span específico. Esse fluxo é chamado de 'Grafana three pillars' e é a base da observabilidade moderna.

```
# docker-compose para Tempo

services:
  tempo:
    image: grafana/tempo:2.6.0
    command: ["-config.file=/etc/tempo.yaml"]
    volumes:
      - ./tempo.yaml:/etc/tempo.yaml:ro
    ports: ["3200:3200"]
    networks: [lab-net]

# Datasource no Grafana
Type: Tempo
URL:  http://tempo:3200

# No Explore, query por service.name
{ resource.service.name = "frontend" } | latency
> 500ms
```

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Arquitetura completa

Em uma arquitetura Grafana completa, temos:

- Grafana — a camada de visualização única
- Prometheus / Mimir — métricas (curto e longo prazo)
- Loki — logs agregados
- Tempo — traces distribuídos
- Pyroscope — continuous profiling (parente mais novo)
- Alloy / Promtail / OTel Collector — coleta unificada

Tudo é open source, todos rodam em Docker, todos se conversam via OTLP ou PromQL.

Caminho de aprendizado após este curso

1. Certificação Grafana Certified Observability Engineer (GCOE) — oficial
2. Praticar com Loki + Tempo em ambiente pessoal
3. Estudar Mimir para métricas em escala
4. Aprender OpenTelemetry (instrumentação)
5. Aprofundar em SRE, SLI/SLO, error budgets

Onde buscar: training.grafana.com, grafana.com/blog, github.com/grafana

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

12 regras de ouro para o dia a dia

1. Nomeie painéis
2. Use variáveis
3. Limite painéis
4. Documente no JSON
5. Versione no Git
6. Alerte no for
7. Roteie por severidade
8. Monitore o monitoramento

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

O que você aprendeu

- Os 3 pilares da observabilidade e como o Grafana unifica tudo
- A subir uma stack completa em Docker Compose em 1 comando
- A criar dashboards com stat, time series, gauge, table, bar, heatmap
- A usar variáveis para tornar dashboards dinâmicos
- A escrever queries PromQL com rate, topk, agregação por label
- A instalar Grafana + Prometheus + Node Exporter em Ubuntu via APT
- A configurar systemd, provisionar datasources e validar a stack
- A configurar alertas Prometheus e Grafana com tempo de espera
- A exportar, importar e versionar dashboards como JSON
- A adicionar Loki e Tempo para logs e traces
- A aplicar boas práticas de produção (for em alertas, versionamento)

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

PRÓXIMOS PASSOS

Para continuar aprendendo

Oficial

Certificação

Comunidade

GitHub

Blog

Slack

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição

Q & A

Perguntas e revisão

Vamos abrir para perguntas.

Temas sugeridos: instalação em produção, observabilidade em Kubernetes, integração com outras ferramentas, carreira em SRE.

Material de apoio:

- Slides deste curso (esta apresentação)
- Caderno de exercícios (PDF — caderno-exercicios.pdf)
- Scripts de laboratório (lab/dia1, lab/dia2, lab/dia3)
- Dashboards pré-configurados (incluídos no compose)

Tudo no pacote entregue pelo instrutor. Material liberado para uso interno, sem redistribuição.

CÓPIA DO ALUNO

OBRIGADO

Sucesso na sua jornada!

Grafana é mais que uma ferramenta — é a forma moderna de olhar para sistemas complexos.

Em 16 horas vocês viram a base completa. Agora é praticar, errar, ajustar, e construir dashboards que ajudem seu time a tomar decisões melhores.

Obrigado pela presença e pelo tempo dedicado. Estarei disponível para dúvidas no email do curso.

— Lyjor

CÓPIA DO ALUNO

Material do aluno — uso pessoal, sem redistribuição